```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.metrics import classification_report, confusion_matrix,
ConfusionMatrixDisplay

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping

import os
from PIL import Image

tf.random.set_seed(42)
np.random.seed(42)

print("GPU:", tf.config.list_physical_devices("GPU"))
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native
Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU
will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
GPU: []
```

```
DATA_DIR=r"D:\dl-workspace\notebooks\datasets\brain_tumor"
print(os.listdir(DATA_DIR))
```

```
['Testing', 'Training']
```

```
for root,dirs,files in os.walk(DATA_DIR):
    print(root,"->",len(files),"files")
```

```
D:\dl-workspace\notebooks\datasets\brain_tumor -> 0 files
D:\dl-workspace\notebooks\datasets\brain_tumor\Testing -> 0 files
D:\dl-workspace\notebooks\datasets\brain_tumor\Testing\glioma_tumor ->
100 files
D:\dl-workspace\notebooks\datasets\brain_tumor\Testing\
meningioma_tumor -> 115 files
D:\dl-workspace\notebooks\datasets\brain_tumor\Testing\no_tumor -> 105
files
D:\dl-workspace\notebooks\datasets\brain_tumor\Testing\pituitary_tumor
-> 74 files
D:\dl-workspace\notebooks\datasets\brain_tumor\Training -> 0 files
D:\dl-workspace\notebooks\datasets\brain_tumor\Training\glioma_tumor -
> 826 files
D:\dl-workspace\notebooks\datasets\brain_tumor\Training\
meningioma_tumor -> 822 files
D:\dl-workspace\notebooks\datasets\brain_tumor\Training\no_tumor ->
395 files
```

```
D:\dl-workspace\notebooks\datasets\brain_tumor\Training\
pituitary_tumor -> 827 files
```

```
train_dir=r"D:\dl-workspace\notebooks\datasets\brain_tumor\Training"
test_dir=r"D:\dl-workspace\notebooks\datasets\brain_tumor\Testing"

IMG_SIZE=(128,128)
BATCH=32

train_ds=keras.utils.image_dataset_from_directory(
    train_dir,
    validation_split=0.1,
    subset="training",
    image_size=IMG_SIZE,
    batch_size=BATCH,
    seed=42,
    label_mode="int"
)
val_ds=keras.utils.image_dataset_from_directory(
    train_dir,
    validation_split=0.1,
    subset="validation",
    seed=42,
    image_size=IMG_SIZE,
    batch_size=BATCH,
    label_mode="int"
)
test_ds=keras.utils.image_dataset_from_directory(
    test_dir,
    image_size=IMG_SIZE,
    batch_size=BATCH,
    label_mode="int",
    shuffle=False
)
```

```
Found 2870 files belonging to 4 classes.
Using 2583 files for training.
Found 2870 files belonging to 4 classes.
Using 287 files for validation.
Found 394 files belonging to 4 classes.
```

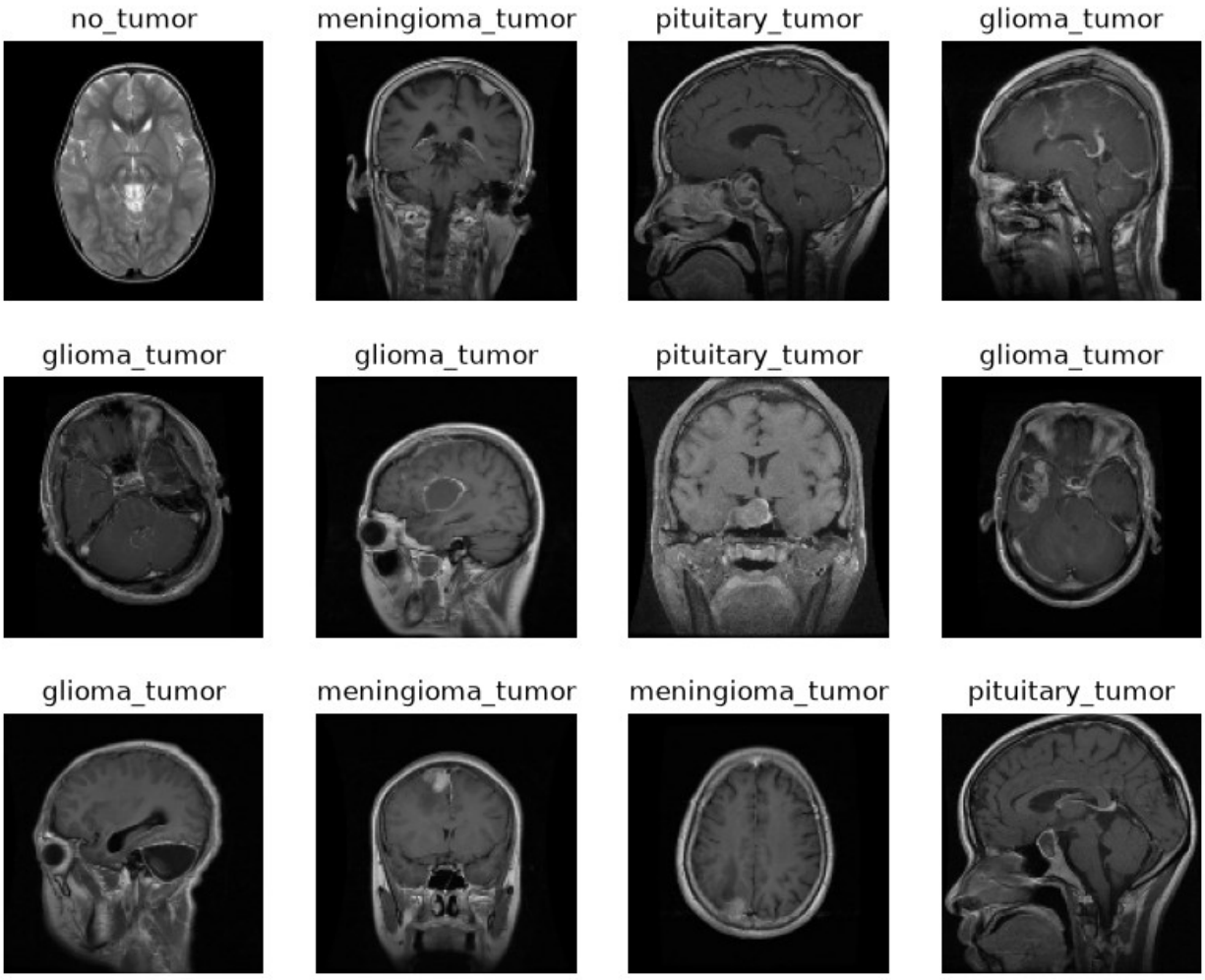
```
class_names=train_ds.class_names
num_class=len(class_names)
print(num_class)
print(class_names)
```

```
4
['glioma_tumor', 'meningioma_tumor', 'no_tumor', 'pituitary_tumor']
```

```
plt.figure(figsize=(10, 8))
```

```
for images, labels in train_ds.take(1):
    for i in range(12):
        plt.subplot(3, 4, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(class_names[labels[i].numpy()])
        plt.axis("off")

plt.show()
```




```
model=keras.Sequential([
    layers.Input(shape=(128,128,3)),
    layers.Rescaling(1.0/255.0),

    layers.Conv2D(32,(3,3), activation="relu"),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64,(3,3),activation="relu"),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(128,(3,3),activation="relu"),
```

```
    layers.MaxPooling2D((2,2)),
    layers.Flatten(),
    layers.Dense(128,activation="relu"),
    layers.Dropout(0.3),
    layers.Dense(4,activation="softmax")
])
model.summary()
```

```
Model: "sequential_2"
```

Layer (type)	Output Shape	Param #
rescaling_2 (Rescaling)	(None, 128, 128, 3)	0
conv2d_6 (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d_6 (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_7 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_7 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_8 (Conv2D)	(None, 28, 28, 128)	73,856
max_pooling2d_8 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten_2 (Flatten)	(None, 25088)	0
dense_4 (Dense)	(None, 128)	3,211,392

```
├─────────────────────────────────────┼─────────────────────────────┼──────────
──────┤
│ dropout_2 (Dropout)                 │ (None, 128)                 │
0 │
├─────────────────────────────────────┼─────────────────────────────┼──────────
──────┤
│ dense_5 (Dense)                     │ (None, 4)                   │
516 │
└─────────────────────────────────────┴─────────────────────────────┴──────────
──────┘

 Total params: 3,305,156 (12.61 MB)

 Trainable params: 3,305,156 (12.61 MB)

 Non-trainable params: 0 (0.00 B)

model.compile(
    loss="sparse_categorical_crossentropy",
    optimizer=Adam(learning_rate=0.001),
    metrics=["accuracy"],
)

history=model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=10,
    verbose=1,
)

Epoch 1/10
81/81 ━━━━━━━━━━━━━━━━━━━━ 13s 140ms/step - accuracy: 0.5768 - loss:
0.9805 - val_accuracy: 0.6899 - val_loss: 0.6332
Epoch 2/10
81/81 ━━━━━━━━━━━━━━━━━━━━ 11s 134ms/step - accuracy: 0.7561 - loss:
0.5853 - val_accuracy: 0.8153 - val_loss: 0.4686
Epoch 3/10
81/81 ━━━━━━━━━━━━━━━━━━━━ 13s 164ms/step - accuracy: 0.8370 - loss:
0.4233 - val_accuracy: 0.8746 - val_loss: 0.3420
Epoch 4/10
81/81 ━━━━━━━━━━━━━━━━━━━━ 15s 188ms/step - accuracy: 0.8773 - loss:
0.3045 - val_accuracy: 0.7979 - val_loss: 0.4661
Epoch 5/10
81/81 ━━━━━━━━━━━━━━━━━━━━ 15s 181ms/step - accuracy: 0.9013 - loss:
0.2524 - val_accuracy: 0.8920 - val_loss: 0.2588
Epoch 6/10
81/81 ━━━━━━━━━━━━━━━━━━━━ 14s 178ms/step - accuracy: 0.9214 - loss:
0.1983 - val_accuracy: 0.8641 - val_loss: 0.3275
Epoch 7/10
81/81 ━━━━━━━━━━━━━━━━━━━━ 15s 183ms/step - accuracy: 0.9586 - loss:
```

```
0.1211 - val_accuracy: 0.9199 - val_loss: 0.2178
Epoch 8/10
81/81 ━━━━━━━━━━━━━━━━━━━━ 16s 192ms/step - accuracy: 0.9655 - loss:
0.0855 - val_accuracy: 0.9164 - val_loss: 0.2177
Epoch 9/10
81/81 ━━━━━━━━━━━━━━━━━━━━ 18s 221ms/step - accuracy: 0.9702 - loss:
0.0833 - val_accuracy: 0.9164 - val_loss: 0.2518
Epoch 10/10
81/81 ━━━━━━━━━━━━━━━━━━━━ 19s 229ms/step - accuracy: 0.9783 - loss:
0.0636 - val_accuracy: 0.9199 - val_loss: 0.2730

test_loss,test_acc=model.evaluate(test_ds)
print(test_loss)
print(test_acc)

y_true=np.concatenate([y.numpy() for x,y in test_ds])
y_pred=model.predict(test_ds).argmax(axis=1)

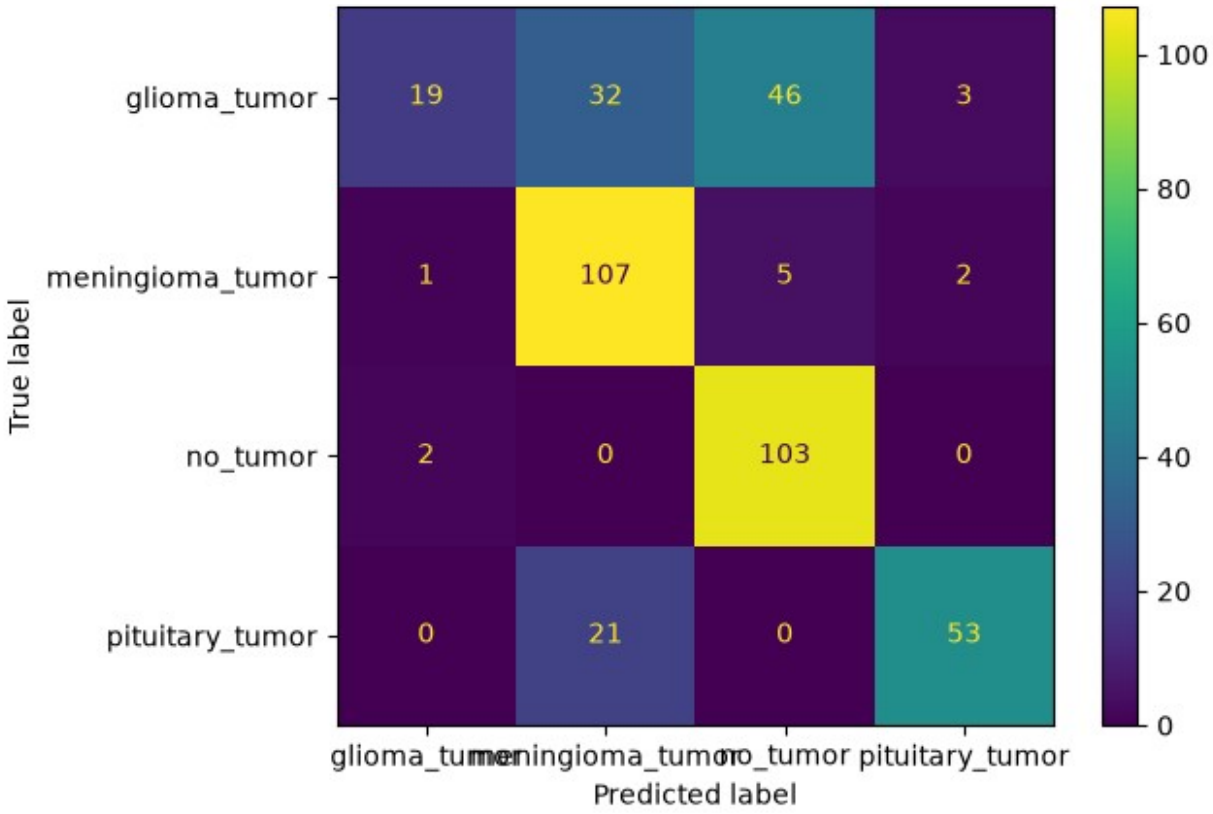
print(classification_report(y_true,y_pred))
ConfusionMatrixDisplay.from_predictions(y_true,y_pred,display_labels=c
lass_names)

13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.7157 - loss:
3.2489
3.2489354610443115
0.7157360315322876
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step
              precision    recall  f1-score   support

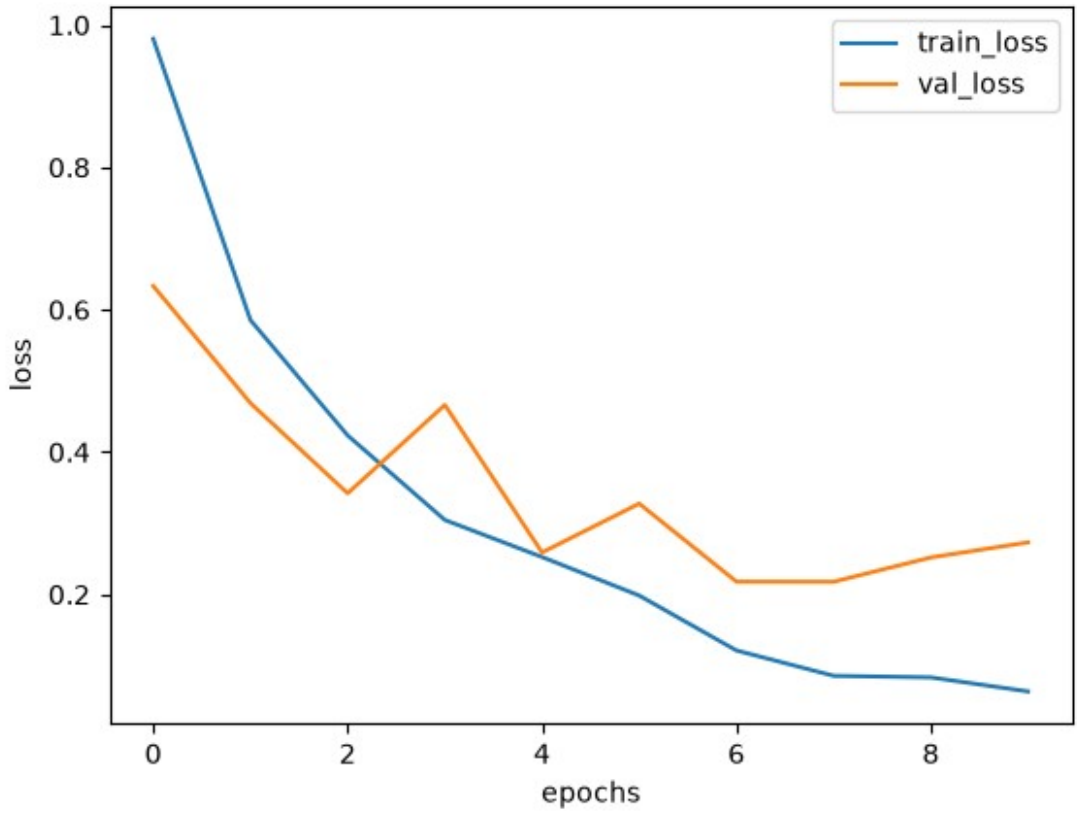
           0       0.86      0.19      0.31       100
           1       0.67      0.93      0.78       115
           2       0.67      0.98      0.80       105
           3       0.91      0.72      0.80        74

    accuracy                           0.72       394
   macro avg       0.78      0.70      0.67       394
weighted avg       0.76      0.72      0.67       394


<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at
0x1d4ff4c27b0>
```




```
plt.plot(history.history["loss"],label="train_loss")
plt.plot(history.history["val_loss"],label="val_loss")
plt.xlabel("epochs")
plt.ylabel("loss")
plt.legend()
plt.show()
```


train_loss
val_loss
1.0
0.8
0.6
0.4
0.2
loss
0
2
4
6
8
epochs